A Glimpse Is Enough: Mask-Aware Sparse-Input Traffic Engineering for LEO Satellite Constellations

Linhui Wei and Guangteng Fan

Abstract—Traffic engineering (TE) is a key control primitive for low-Earth-orbit (LEO) satellite constellations, whose topologies and traffic change continuously as satellites move. Existing learning-based TE systems — for terrestrial and satellite networks alike — assume that the complete traffic matrix is measured and delivered to the controller at every decision epoch. This assumption rarely holds in practice: network-wide measurement competes with user traffic for scarce onboard resources, and the controller typically observes only a sparse subset of demands. We present SiTE, a learning-based TE system that allocates traffic for *all* active demands directly from sparse observations. SiTE represents unmeasured demands with learnable mask embeddings instead of zeros, gates message passing so that placeholders never contaminate link-state estimates, and recovers missing information from the recent observation history through a Transformer fused with spatial embeddings via per-demand cross-attention — all trained end-to-end against the TE objective without an explicit completion stage. Demand pruning and size-invariant weights further allow a model trained once to be reused as the topology and the active demand set drift, without retraining. On a 1,584-satellite Starlink-like constellation with real traffic dynamics, SiTE keeps the realized maximum link utilization within [TODO: XX]% of the fully-observed optimum with only [TODO: XX]% observability, outperforming zero-filling and two-stage complete-then-optimize baselines by up to [TODO: XX]%, at [TODO: XX] ms inference per snapshot.

Index Terms—Satellite networks, traffic engineering, sparse measurement, graph neural networks, reinforcement learning.

1 Introduction

Low-Earth-orbit (LEO) satellite constellations are rapidly scaling toward tens of thousands of satellites [1], [2], [3], carrying user traffic with pronounced spatial and temporal fluctuations over inter-satellite links (ISLs) of limited capacity [4], [5]. Without global coordination, shortest-path or purely distributed routing concentrates traffic on a few congested ISLs, leaving network capacity underutilized. Traffic engineering (TE) — centrally optimizing how each demand is split across candidate paths — has therefore become a key control primitive for satellite networks, just as it has been for cloud wide-area networks (WANs) over the past decade [6], [7], [8], [9].

A recent line of learning-based TE systems has made centralized control practical at scale: by replacing online optimization solvers with neural models, they cut allocation time from minutes to milliseconds on WANs [9], [10], [11] and, most recently, on satellite constellations [12]. These systems, however, share a silent premise: the *complete* traffic matrix — the demand between every pair of nodes — is assumed to be measured and delivered to the controller at every decision epoch.

This premise rarely holds in LEO constellations. Network-wide measurement requires every satellite to meter, aggregate, and downlink fine-grained flow statistics through bandwidth-constrained telemetry channels, competing with user traffic and consuming scarce onboard resources; measurement gaps due to intermittent ground contacts and equipment degradation are the norm rather than the exception. In practice, the controller observes only a *sparse* traffic matrix — a subset of demands measured at each epoch. Naive workarounds fall short on both ends: filling unobserved entries with zeros misleads the allocator — it leaves no headroom on the links that unmeasured traffic actually traverses, inflating the realized maximum link utilization (MLU), and under throughput objectives it systematically starves unmeasured demands — while two-stage "complete-then-optimize" pipelines propagate reconstruction errors into allocation decisions. The state-of-the-art sparse-TE approach [13] fills unobserved entries with zeros and compensates with synthetic training augmentation, but it targets terrestrial networks with static topologies — leaving sparse-observation TE for dynamic satellite networks unaddressed.

Designing a TE system that operates on sparse observations of satellite traffic raises three challenges. **(C1) Representing the unobserved.** Unobserved demands are not zero; the model must distinguish "no traffic" from "not measured", infer missing volumes from history, and prevent placeholder values from contaminating the shared network representation during message passing. **(C2) Perpetual dynamics.** Satellites move: topologies, candidate paths, and even the set of active demand pairs drift over time. Retraining per snapshot is infeasible; the model must be trained once and reused across snapshots unseen in training. **(C3) Constellation scale.** A 1,584-satellite constellation induces 2.5 million demand pairs and tens of millions of path variables if modeled exhaustively — beyond the memory

• *L. Wei and G. Fan are with the National Innovation Institute of Defense Technology, Academy of Military Science, Beijing, China (e-mail: weilinhui998@163.com; fanguangteng@163.com).*

Manuscript received XXX; revised XXX. (Corresponding author: Guangteng Fan.)

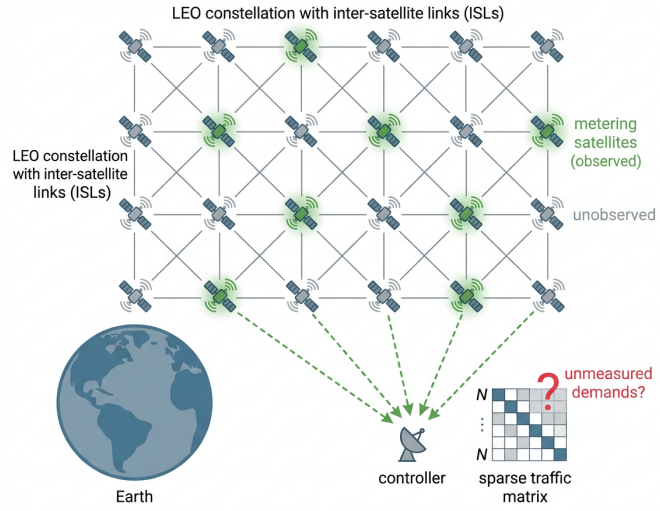

Fig. 1. Sparse traffic observation in an LEO constellation: only a subset of satellites meters traffic, so the controller receives a sparse traffic matrix yet must allocate all demands.

and latency budget of any online system.

We present SiTE (Sparse-input Traffic Engineering), a learning-based TE system for LEO constellations that allocates traffic directly from sparse observations. SiTE builds on the flow-centric GNN and multi-agent RL backbone of Teal [9] and augments it with three sparse-aware components. First, unobserved demands are represented by *learnable mask embeddings* rather than zeros, letting the model learn a prior for missing traffic (C1). Second, a *mask-aware gated GNN* blocks message passing from unobserved path nodes to link nodes — preventing placeholder pollution of link utilization estimates — while still letting unobserved demands sense link states in the reverse direction (C1). Third, a *Transformer temporal encoder* summarizes the recent history of sparse observations and fuses it with spatial embeddings via per-demand cross-attention, recovering missing information from temporal correlation (C1). For deployment at scale, SiTE prunes the demand set to pairs with observed traffic — $\sim$400$\times$ fewer variables on a 1,584-satellite Starlink-like constellation — and keeps all model weights independent of the demand-set size, so a model trained once transfers across topology snapshots and drifting demand sets without retraining (C2, C3). Allocations are finally refined by a parallelizable ADMM step to repair residual capacity violations.

In summary, this paper makes the following contributions:

- We identify sparse traffic observability as a fundamental yet unaddressed constraint for satellite TE, and formulate TE from sparse observations over dynamic constellations (sections 2 and 3).
- We design SiTE, which combines learnable mask embeddings, mask-aware gated message passing, and temporal-spatial cross-attention to allocate traffic directly from sparse inputs, without an explicit completion stage (section 4).
- We show how demand pruning and size-invariant weights enable train-once, zero-retraining deployment across topology snapshots and drifting demand sets (sections 4 and 5).
- On a 1,584-satellite Starlink-like constellation with real topology dynamics, SiTE keeps the realized MLU within [TODO: XX]% of the fully-observed optimum with only [TODO: XX]% observability — and retains [TODO: XX]% of the optimum under the throughput objective — outperforming zero-filling and two-stage baselines by [TODO: XX–XX]%, with [TODO: XX] ms inference per snapshot (section 5).

2 Related Work

2.1 ML-Driven Traffic Engineering for WANs

Traffic engineering has long been formulated as a constrained optimization problem [14], [15] solved by linear programming (LP), whose runtime scales poorly with network size. SMORE [16] decouples robust oblivious path selection from online rate adaptation, while NCFlow [17] and POP [18] accelerate LP by problem decomposition, trading allocation quality for speed. A recent line of work replaces online solvers with learned models [19]: DOTE [10] directly maps historical traffic matrices to split ratios with stochastic optimization; Teal [9] combines a flow-centric GNN, multi-agent RL, and ADMM fine-tuning to achieve near-optimal allocations with orders-of-magnitude speedups; FIGRET [20] enhances robustness against traffic uncertainty at per-flow granularity. To survive topology changes without retraining, HARP [11] enforces invariances to node permutation and tunnel reordering, transferring across topologies unseen in training. More recently, large language models have been explored as general-purpose traffic planners [21]. However, all of these systems assume the *complete* traffic matrix is available at decision time — an assumption that rarely holds in large-scale satellite networks, where network-wide measurement is prohibitively expensive.

2.2 TE and Routing for Satellite Networks

LEO mega-constellations pose unique challenges for TE: topologies change every few tens of milliseconds as satellites move, and control decisions must be produced within stringent latency budgets [1], [2], [4]. Classical approaches absorb mobility through topology abstractions — the virtual node model binds logical nodes to earth-fixed footprints so that upper-layer routing observes a quasi-static grid [22]. Optimization-based satellite TE has explored segment routing [23], distributed clustering that optimizes elephant flows separately [24], and supervised learning for distributed split-ratio prediction [25]. SaTE [12] is the state-of-the-art learning-based TE for satellite constellations: it formulates a heterogeneous graph over satellites, demands, paths, and links, prunes zero-traffic relations to make training tractable on a single GPU, and selects representative topologies via Graph2Vec and DPP sampling so that one trained model generalizes across topology snapshots with millisecond inference. In parallel, FNC [26], [27] targets dynamic traffic allocation for earth-observation constellations, replacing RL with end-to-end imitation learning and substituting iterative constraint solvers with normalization-based feasibility layers. Notably, FNC's imitation learning constructs its

TABLE 1
Comparison with the closest prior systems.

	Teal [9]	SaTE [12]	TEST [13]	FNC [26]	**Ours**
Satellite scenario	×	✓	×	✓	✓
Dynamic topology	×	✓	×	✓	✓
Sparse observation	×	×	✓	×	✓
Temporal modeling	×	×	✓	×	✓
Mask-aware model	×	×	×	×	✓
Demand-set drift	×	partial	×	×	✓

expert supervision by solving an offline optimum over the *fully realized* demands of each episode — a paradigm that fundamentally requires complete post-hoc observability, which is unavailable under sparse measurement. Both SaTE and FNC, like their WAN counterparts, assume the traffic matrix is fully observable at decision time.

2.3 TE with Incomplete Traffic Observations

Network-wide traffic measurement is expensive: collecting a complete traffic matrix requires instrumenting all ingress nodes at fine time granularity, which is particularly onerous on resource-constrained satellite platforms. A classical workaround is a two-stage pipeline — first estimate or complete the traffic matrix via network tomography [28], [29], matrix completion, or compressive sensing [30], then optimize routing on the estimate — but reconstruction errors propagate to and degrade the downstream allocation. TEST [13] is, to our knowledge, the first end-to-end approach that learns routing policies directly from sparsely measured traffic matrices, integrating a Transformer over historical sparse sequences with a GCN over the topology; it fills unobserved entries with zeros, compensates with synthesized training augmentation, and targets terrestrial networks with static topologies. In the graph learning literature, message passing under missing node features has been addressed via teacher–student distillation [31], but such techniques have not been applied to network TE. No prior work, to our knowledge, performs TE from sparse traffic observations in satellite networks, where the difficulty is compounded by time-varying topologies and drifting demand sets.

2.4 Positioning

Our work sits at the intersection of these three lines (table 1). Unlike SaTE and FNC, which assume fully observable demands, we perform TE from sparse observations (10%–70% observability). Unlike TEST, which fills unobserved entries with zeros and targets static terrestrial networks, we replace missing entries with learnable mask embeddings, gate their message passing, and handle LEO dynamics — time-varying topologies and drifting demand-pair sets — via demand pruning with size-invariant model weights, enabling train-once, zero-retraining deployment.

3 Model and Problem Formulation

3.1 Network and Traffic Model

We model the constellation at control epoch t as a directed graph $G^t = (V, E^t)$, where V is the set of N satellites and E^t the set of inter-satellite links (ISLs), each link e with capacity c_e. Time is slotted into control epochs; within an epoch the topology snapshot is fixed, while across epochs both E^t and the traffic change as satellites move.

TABLE 2
Main Notations.

Symbol	Definition
$G^t = (V, E^t)$	constellation graph at epoch t; c_e link capacity
D^t, d_i	demand matrix; volume of demand i
$\mathcal{K}^t$	active demand set at epoch t
κ, P_i	candidate paths per demand; path set of demand i
r_i, $f_{i,k}$	split ratios of demand i; flow on path $p_{i,k}$
M, ρ	binary observation mask; observability ratio
O^t, H^t	sparse observation $D^t \odot M$; last-L history
ϕ	learnable mask embedding for unmeasured demands
A, $\tilde{A}$	bipartite adjacency; its mask-gated variant
H_S, H_T, H_F	spatial, temporal, and fused embeddings
π_θ, R	TE policy network; ground-truth reward

Traffic is described by a demand matrix $D^t \in \mathbb{R}^{N\times N}$, where d_i denotes the volume of the i-th source–destination pair. Since satellite traffic is geographically concentrated, we maintain an *active demand set* $\mathcal{K}^t$ — the pairs with nonzero demand in recent history (section 4) — and only these are modeled. Each demand $i \in \mathcal{K}^t$ is served by κ pre-selected candidate paths $P_i = \{p_{i,1}, \ldots, p_{i,\kappa}\}$ ($\kappa = 4$ edge-disjoint shortest paths in our implementation). The TE decision for demand i is a split-ratio vector $r_i \in \mathbb{R}^\kappa$ with $r_{i,k} \geq 0$ and $\sum_k r_{i,k} \leq 1$, allocating flow $f_{i,k} = r_{i,k} \cdot d_i$ to path $p_{i,k}$; the residual $1-\sum_k r_{i,k}$ represents intentionally unserved traffic. Main notations are summarized in table 2.

3.2 TE with Complete Observation

With the complete demand matrix available, TE is the classic path-based allocation problem, for which two objectives dominate the literature. *Load balancing* minimizes the maximum link utilization (MLU) — the standard objective in satellite TE, where hot ISLs over populated regions dictate quality of service [12], [23], [24]:

$$\min_{\{r_{i,k}\}} \quad \theta \tag{1}$$

$$\text{s.t.} \quad \frac{1}{c_e} \sum_{(i,k):\, e\in p_{i,k}} r_{i,k} d_i \leq \theta, \qquad \forall e \in E^t, \tag{2}$$

$$\sum_{k=1}^{\kappa} r_{i,k} = 1, \qquad \forall i \in \mathcal{K}^t, \tag{3}$$

$$r_{i,k} \geq 0, \qquad \forall i, k, \tag{4}$$

where (3) enforces flow conservation: every demand must be fully served, so θ cannot be lowered by simply withholding traffic. *Throughput maximization* instead maximizes the total satisfied demand, permitting demands to be served only partially when capacity is scarce (total-flow TE [9],

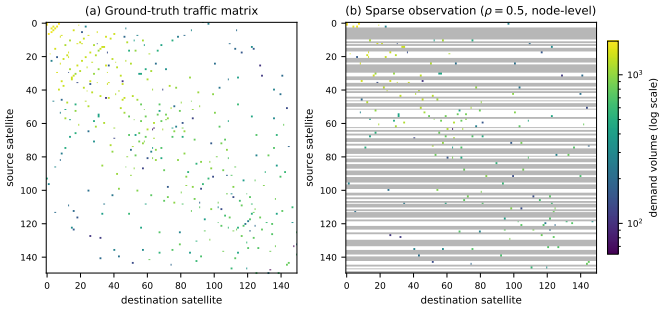

Fig. 2. A ground-truth traffic matrix from the Starlink 22×72 trace (150 busiest satellites shown) and its node-level sparse observation: white cells carry no traffic, gray rows are unmeasured source satellites, and colored cells are observed demands.

[17]):

$$\max_{\{f_{i,k}\}} \quad \sum_{i\in\mathcal{K}^t}\sum_{k=1}^{\kappa} f_{i,k} \tag{5}$$

$$\text{s.t.} \quad \sum_{k=1}^{\kappa} f_{i,k} \le d_i, \qquad \forall i \in \mathcal{K}^t, \tag{6}$$

$$\sum_{(i,k):\, e\in p_{i,k}} f_{i,k} \le c_e, \qquad \forall e \in E^t, \tag{7}$$

$$f_{i,k} \ge 0, \qquad \forall i,k. \tag{8}$$

Both problems are linear programs solvable to optimality, but at constellation scale their solve time far exceeds the control epoch [12], and — more fundamentally for this paper — the link-load terms in (2) and (7), as well as the per-demand cap (6), all require knowing d_i for *every* active demand. We treat MLU minimization as the primary objective throughout this paper, following the satellite TE literature, and report throughput maximization as a secondary instantiation to show that our design is objective-agnostic.

3.3 TE with Sparse Observation

3.3.1 Observation Model

At each epoch only a subset of demands is measured. A binary mask $M \in \{0,1\}^{|\mathcal{K}^t|}$ marks demand i as measured ($M_i = 1$) or not ($M_i = 0$), and the controller receives the sparse observation

$$O^t = D^t \odot M, \quad \text{together with } M \text{ itself}, \tag{9}$$

where $\odot$ is the element-wise product. Figure 2 contrasts a ground-truth traffic matrix from our Starlink trace with its node-level sparse observation. We consider two measurement granularities: *flow-level*, where a fraction ρ of demand pairs is sampled, and *node-level*, where a fraction ρ of satellites meter all their outgoing demands — the latter matching deployments where measurement capability resides on a subset of satellites. The controller additionally retains the last L sparse observations $H^t = (O^{t-L+1}, \ldots, O^t)$.

3.3.2 Problem Statement

A key observation makes decision-making under unknown demands physically executable: the decision variables are split *ratios*, not absolute volumes. Once installed in the forwarding plane, r_i is applied multiplicatively to whatever traffic actually arrives, so a decision for demand i requires no knowledge of d_i. Sparse-observation TE then seeks a policy π that maps the observable state to split ratios for *all* active demands,

$$\{r_i\}_{i\in\mathcal{K}^t} = \pi\left(H^t, M, G^t\right), \tag{10}$$

maximizing the *ground-truth* objective. Let $f_{i,k}(\pi) = \tilde{r}_{i,k}\cdot d_i$ denote the flow actually delivered on path $p_{i,k}$, where $\{\tilde{r}_i\}$ are the policy's ratios after a feasibility-restoration step (needed only for the throughput objective, realized by ADMM refinement and rounding in section 4.6; under MLU the ratios are directly feasible by (3)). Formally, for MLU minimization

$$\min_{\pi}\ \mathbb{E}_t\Big[\max_{e\in E^t}\frac{1}{c_e}\sum_{(i,k):\, e\in p_{i,k}} f_{i,k}(\pi)\Big], \tag{11}$$

and, for the throughput variant,

$$\max_{\pi}\ \mathbb{E}_t\Big[\sum_{i\in\mathcal{K}^t}\sum_{k=1}^{\kappa} f_{i,k}(\pi)\Big], \tag{12}$$

where the expectations are over traffic and topology dynamics. Note that the link loads in (11) are evaluated with the true demands d_i: an unmeasured demand still consumes real capacity even though the policy could not see it, so misjudging it inflates the realized MLU. The problem is a partially observable sequential decision problem: unlike (1)–(8), where the same D^t appears in both the decision and the constraints, here the decision variables and the evaluation live in different information sets — the policy acts on (O, M) only, while feasibility and performance are settled by the hidden D. Intuitively, the policy is graded against an exam it never gets to read in full.

3.4 Solution Paradigm

Three solution families exist for (11) and (12). *Solve-with-zeros* plugs O directly into the LP (1)–(8), implicitly assuming unmeasured demands are zero; the resulting policy leaves no headroom on the links that unmeasured traffic actually traverses, so the realized MLU can be far above the optimum (and, under the throughput objective, unmeasured demands are structurally starved). *Two-stage* approaches first estimate $\hat{D}$ from (H, M) — via interpolation, tensor completion, or compressive sensing [28], [29], [30] — then solve the LP on $\hat{D}$; estimation is optimized for reconstruction error, which is misaligned with allocation quality, and errors propagate un-damped into the allocation. We instead adopt *end-to-end learning*: parameterize π_θ as a neural network and optimize θ directly against the TE objective,

$$\max_{\theta}\ \mathbb{E}\big[R\big(\pi_\theta(H, M, G),\, D\big)\big], \tag{13}$$

where R is the TE objective evaluated on the true D — negative realized MLU for (11), or total satisfied demand for (12). Since R is non-differentiable through the capacity-constrained network and the true D is available only as a training-time signal, we optimize (13) with multi-agent reinforcement learning — each demand acts as an agent

Algorithm 1 SiTE inference at control epoch t

Require: sparse history H^t, mask M, topology G^t, weights $\theta^\star$
Ensure: split ratios $\{r_i\}_{i\in\mathcal{K}^t}$
1: $\mathcal{K}^t \leftarrow$ active demand set from recent history ▷ pruning, section 4.5
2: rebuild bipartite graph and gated adjacency $\tilde{A}$ from $\mathcal{K}^t, M$ ▷ eq. (15)
3: $x_i \leftarrow M_i d_i + (1 - M_i)\phi$ for all i ▷ mask embedding, eq. (14)
4: $H_S \leftarrow \text{GATEDFLOWGNN}(x, \tilde{A})$ ▷ spatial embeddings
5: $H_T \leftarrow \text{TEMPORALENCODER}(H^t)$ ▷ temporal embeddings
6: $H_F \leftarrow \text{CROSSATTENTION}(H_T, H_S)$ ▷ eq. (16)
7: $\{r_i\} \leftarrow \text{POLICYHEAD}([H_S \,\|\, H_F])$
8: $\{r_i\} \leftarrow \text{ADMMREPAIR}(\{r_i\}, G^t)$ ▷ section 4.6
9: **return** $\{r_i\}$

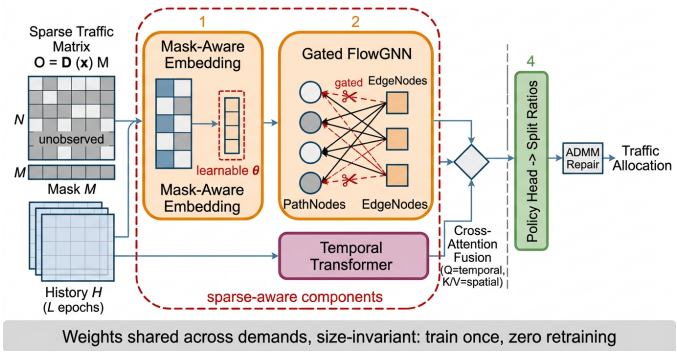

Fig. 3. SiTE overview: a sparse observation is embedded with mask awareness, processed by a gated flow-centric GNN in parallel with a temporal encoder, fused via per-demand cross-attention, and mapped to split ratios refined by ADMM.

sharing θ, receiving a COMA-style marginal-contribution reward (section 4) — followed by a lightweight ADMM refinement at inference to repair residual violations of (7). This paradigm makes the observation model (9) a property of the *input*, and the optimization target (12) a property of the *training signal*, cleanly separating what the policy can see from what it is optimized for.

4 SiTE Design

4.1 Overview

SiTE solves the sparse-observation TE problem of section 3.3 in four stages, as illustrated in fig. 3. Given the sparse observation (O^t, M) and history H^t, (i) unmeasured demands are represented by a learnable mask embedding (section 4.2); (ii) a mask-aware gated GNN propagates information between candidate paths and links without letting placeholders contaminate link states (section 4.3); (iii) a Transformer encodes the observation history and fuses it with spatial embeddings via per-demand cross-attention (section 4.4); (iv) a shared policy head outputs split ratios per demand, refined by ADMM (section 4.6). All learnable weights are shared across demands and independent of the demand-set size, which section 4.5 exploits for zero-retraining deployment under constellation dynamics. Algorithm 1 summarizes one inference epoch.

4.2 Mask-Aware Demand Representation

A sparse observation is fundamentally ambiguous: a zero entry may mean "no traffic" or "not measured". Zero-filling — the de-facto treatment in prior sparse TE [13] — collapses the two cases and systematically biases the allocator toward starving unmeasured demands. SiTE instead assigns each unmeasured demand a *learnable mask embedding* $\phi \in \mathbb{R}^\kappa$ (one scalar per candidate path):

$$x_i = M_i \cdot d_i + (1 - M_i) \cdot \phi, \tag{14}$$

so the model learns a data-driven prior for missing traffic during end-to-end training, and — critically — the input itself distinguishes "not measured" from "zero". Learnable imputation has proven effective for incomplete graph features [31]; we bring it to TE, where the imputed value directly steers resource allocation. The embedding is trained jointly with the policy under the TE objective (13) rather than a reconstruction loss, so ϕ converges to values that maximize allocation quality, not reconstruction accuracy.

4.3 Gated Message Passing

SiTE inherits Teal's flow-centric graph [9]: candidate paths and physical links form a bipartite structure where a PathNode connects to the EdgeNodes it traverses, and message passing alternates between them with GCN-style normalized aggregation [32]. Under sparse observation, however, symmetric message passing is harmful: placeholder values on unmeasured PathNodes would propagate into EdgeNode embeddings and corrupt the network-wide estimate of link utilization — a phenomenon consistent with the feature–structure interference observed in incomplete graphs [31]. We therefore gate the propagation *asymmetrically*. Let A be the normalized bipartite adjacency; we zero out entries that carry path-to-edge messages from unmeasured paths while keeping all edge-to-path entries:

$$\tilde{A}[e,p] = M_p \cdot A[e,p], \qquad \tilde{A}[p,e] = A[p,e], \tag{15}$$

where M_p is the demand-level mask lifted to path p. Unmeasured demands thus remain *listeners*: they sense link states — which are shaped by measured traffic — but inject no fabricated volume into them. The gate is a fixed binary function of the mask with no extra parameters, adding zero inference overhead. The asymmetric gate yields a formal isolation property:

Proposition 1 (Placeholder isolation). Under the gated propagation (15), the embeddings of all EdgeNodes and of all PathNodes belonging to measured demands are independent of the mask embedding ϕ; ϕ influences only the allocations of unmeasured demands.

Proof: Consider first the spatial stream, by induction on GNN layers. At layer 0, only unmeasured PathNodes carry ϕ. At layer l, an EdgeNode aggregates path-to-edge messages with weights $\tilde{A}[e,p] = M_p A[e,p]$, which vanish for every unmeasured path; hence EdgeNode embeddings

depend only on measured-path inputs and their own history, none of which contains ϕ. A measured PathNode aggregates EdgeNode embeddings (ϕ-free by the above) and its own ϕ-free input; the per-demand layers mix only paths of the same demand, which share the same mask value. The temporal and fusion streams preserve the property: the temporal encoder acts per path on the raw observation history H^t, which contains no ϕ by construction, and the cross-attention of (16) operates strictly within each demand, so the fused features of a measured demand combine only its own ϕ-free temporal and spatial embeddings. Thus ϕ never enters any ϕ-free branch, and only unmeasured PathNodes — and consequently their own split ratios — depend on ϕ. □

In other words, learning a poor prior ϕ can at worst misallocate the unmeasured demands themselves, but can never corrupt the allocations of measured traffic — a robustness guarantee that zero-filling with ungated propagation does not provide.

4.4 Temporal-Spatial Fusion

Satellite traffic exhibits strong short-term temporal correlation; the recent history of sparse observations therefore carries recoverable information about currently missing entries. SiTE encodes, for each candidate path, its observation sequence over the last L epochs with a lightweight Transformer [33]: scalar volumes are log-compressed — which we find essential for training stability — and projected to d_{model} dimensions, summed with learnable positional embeddings, and passed through a Transformer encoder; the representation at the latest step forms the temporal embedding H_T. Rather than concatenating temporal and spatial features naively, we fuse them with *per-demand cross-attention*: within each demand, temporal embeddings of its κ candidate paths act as queries attending to the κ spatial (GNN) embeddings H_S as keys and values,

$$H_F = \text{softmax}\left(\frac{W_q H_T \,(W_k H_S)^\top}{\sqrt{d}}\right) W_v H_S, \qquad (16)$$

and the fused features $[H_S \,\|\, H_F]$ feed the policy head. Intuitively, the temporal stream decides *which* spatial evidence to trust for each path: when the current entry is missing, attention shifts toward paths and links whose states corroborate the historical pattern. The whole module adds [TODO: XX] ms to inference on our largest constellation (section 5).

4.5 Scaling to Dynamic Constellations

4.5.1 Demand Pruning

Modeling all $N(N-1)$ pairs of a 1,584-satellite constellation yields 2.5M demands and tens of millions of path variables — infeasible for online control. Satellite traffic, however, is geographically concentrated: only 0.25% of pairs ever carry traffic in our dataset. SiTE instantiates PathNodes only for the demand-pair union observed in historical traffic matrices, shrinking the graph by $\sim 400\times$ with no loss of served traffic, in the same spirit as SaTE's traffic pruning [12].

4.5.2 Zero-Retraining under Drift

Pruned demand sets are not static: as satellites move, the active pair set drifts (we measure $\sim 10\%$ turnover across our trace, section 5). Rather than retraining, SiTE exploits a structural property of its architecture: every learnable component — GNN layers, mask embedding, temporal encoder, fusion and policy heads — is shared across demands and independent of the demand-set size. The demand set, candidate paths, and the bipartite graph are runtime *inputs*, not parameters. At each control epoch the controller rebuilds the graph from the current active set — a millisecond-scale index operation on +Grid constellations, where candidate paths depend only on the orbital displacement between endpoints and reduce to $O(N)$ path templates — and reuses the trained weights as-is. Demands outside the modeled set — necessarily sporadic mice flows — fall back to shortest-path routing until the next set refresh. This reuse is justified by the following structural property:

Proposition 2 (Size invariance and permutation equivariance). The parameter set of SiTE is independent of $|\mathcal{K}^t|$, N, and $|E^t|$, and the mapping from inputs to split ratios is equivariant to any permutation of the demand indices.

Proof: Every learnable component is a per-node or per-demand operator with shared weights: the mask embedding ϕ and temporal encoder act on each path token, the GNN layers combine node-wise linear maps with sparse aggregation, the fusion and policy heads act within each demand. None of their parameter shapes involves $|\mathcal{K}^t|$, N, or $|E^t|$. Permuting demand indices permutes the rows of every input and of the bipartite adjacency consistently; since shared node-wise operators commute with permutations and sparse aggregation is index-based, all intermediate embeddings — and hence the outputs — are permuted identically. □

4.6 Training and Constraint Repair

SiTE trains with the multi-agent RL scheme of Teal [9]: each demand is an agent sharing the policy network. The policy outputs a Gaussian mean per path, and actions are sampled during training. To assign credit, each agent receives a COMA-style [34] counterfactual reward that estimates its marginal contribution to the global objective (13): for demand i,

$$A_i = \mathbb{E}_{\tilde{r}_i \sim \pi'}\big[R(r_{-i}, r_i) - R(r_{-i}, \tilde{r}_i)\big], \qquad (17)$$

where r_{-i} denotes the allocations of all other demands and $\tilde{r}_i$ is a counterfactual action resampled from a reference distribution π'; the policy gradient weights each agent's log probability by A_i. Algorithm 2 summarizes offline training. Two points deserve emphasis. First, the reward is computed against *ground-truth* demands during offline training — sparse observation constrains the policy input, never the training signal — which is exactly what a simulator or historical trace provides. Second, we deliberately retain reward-driven RL rather than imitation learning [26]: imitation learning requires expert allocations solved from *fully realized* demands of each episode, a supervision signal that is unavailable when even post-hoc observation is sparse; RL needs only the reward, which the network itself provides. At

Algorithm 2 Offline training of SiTE

Require: historical trace $\{(D^t, G^t)\}$, mask M, learning rate η
Ensure: trained weights $\theta^\star$ (incl. mask embedding ϕ)
build demand set $\mathcal{K}$ and graph from training-slice TMs
for each epoch **do**
  for each snapshot t in the training slice **do**
    form sparse input (H^t, M); sample actions $r \sim \pi_\theta(\cdot \mid H^t, M, G^t)$
    compute counterfactual rewards $\{A_i\}$ via (17) **using the true** D^t
    $\theta \leftarrow \theta + \eta \sum_i A_i \nabla_\theta \log \pi_\theta(r_i)$
  end for
end for
return $\theta^\star$

inference, the MLU objective requires no repair — the pure-softmax ratios satisfy flow conservation (3) by construction — while under the throughput objective allocations are refined by two parallelizable ADMM [35] iterations to repair residual violations of the capacity constraint (7), adding less than 1 ms.

4.7 Complexity Analysis

Let $D = |\mathcal{K}^t|$ be the number of active demands, κ the candidate paths per demand, E the number of ISLs, and $\bar{\ell}$ the average path length. The gated FlowGNN performs message passing over the bipartite graph whose edge count is $O(D\kappa\bar{\ell})$; with Λ layers its cost is $O(\Lambda D\kappa\bar{\ell}d)$ for hidden width d. The temporal encoder applies a Transformer of length L independently to each of the $D\kappa$ path tokens, costing $O(D\kappa L^2 d)$, and the per-demand cross-attention over κ paths adds $O(D\kappa^2 d)$. Since κ, L, $\bar{\ell}$, and d are small constants fixed by design, the end-to-end inference cost is *linear* in the number of active demands, $O(D)$, and all operations are batched across demands on the GPU. Crucially, the parameter count is independent of D, N, and E: the same weights apply to any demand set, which is what enables zero-retraining reuse (section 4.5). In our configuration the entire model has merely 4,468 trainable parameters — small enough to be replicated on resource-constrained satellite platforms. Demand pruning reduces D from $N(N-1)$ to the observed support — roughly $400\times$ on our constellation — turning an otherwise intractable problem into one solved in milliseconds (section 5.5).

5 Evaluation

5.1 Setup

Constellation. Our testbed is a Starlink-like Walker constellation interconnected in a +Grid pattern, with parameters listed in table 3; the orbital dynamics are derived from the Satellite Tool Kit (STK). Table 4 compares its graph characteristics with the terrestrial topologies used in prior TE studies [9], which we additionally employ for generalization experiments: the constellation combines a large node count with a long average path length (23.51 hops) and diameter (47), making capacity conflicts between demands substantially more frequent than in terrestrial WANs.

TABLE 3
Simulation parameters of the Starlink constellation.

Parameter	Value
Height of LEO satellites	550 km
Constellation configuration	Walker
Total number of satellites	1,584
Number of orbital planes	72
Satellites per plane	22
Inclination of orbits	55°
Phase factor	3
Inter-satellite links	6,336

TABLE 4
Characteristics of evaluated network topologies (computed from the topology data).

Topology	Nodes	Edges	Avg. path len.	Diameter
B4	12	38	2.32	5
UsCarrier	158	378	12.09	35
Kdl	754	1,790	22.73	58
ASN	1,739	8,558	3.24	8
Starlink	1,584	6,336	23.51	47

Traffic. Traffic matrices are generated by random city sampling [36]: cities are drawn with probability proportional to their population (or GDP), each sampled city pair is attached to its nearest satellites, and per-pair flows are aggregated into satellite-level demands. We use the population-based trace as the default: 101 consecutive snapshots (one every 5 minutes) that capture real geographic demand distribution and temporal drift ($\sim$10% demand-pair turnover across the trace); the GDP-based variant, whose traffic is thinner but spread over $\sim$20$\times$ more demand pairs (table 5), is used in the sensitivity study. ISL capacity is calibrated so that the fully-observed optimum leaves moderate congestion (satisfied ratio $\approx$0.87), keeping TE decisions non-trivial. Snapshots 0–79 are used for training, 80–89 for validation, and 90–100 for testing — a strict temporal split with no leakage. Table 5 summarizes the trace; fig. 4 illustrates its two properties that SiTE exploits: strong short-term temporal correlation of per-satellite volumes, and pronounced geographic concentration — satellites over populated regions carry orders of magnitude more traffic than those over oceans, which is what makes demand pruning (section 4.5) effective.

Metrics and baselines. Our primary metric is the *realized MLU*, evaluated against ground-truth demands. Following TEST [13], we report it as the normalized performance ratio $PR = U/U_{\text{opt}}$, where U_{opt} is the optimal MLU obtained by solving the LP (1)–(4) on the true traffic matrix ($PR \geq 1$, lower is better); MLU alone can be gamed by under-serving traffic, which our flow-conservation formulation (3) rules out by construction. As the secondary objective we report the *satisfied demand ratio* (allocated flow after ADMM and rounding over total demand). Both metrics and per-snapshot inference latency are averaged over three seeds (mean $\pm$ std). Following the taxonomy of section 3.4, we compare against *Zero-fill* (unobserved entries set to zero, no mask-aware modules), *Mean-interp.* (two-stage complete-then-optimize), and *Teal-full* (original Teal with the complete matrix, an upper reference).

TABLE 5
Statistics of the traffic traces (real data, Mbps).

Metric	Population-based	GDP-based
Snapshots (5-min interval)	101	101
Nonzero demand, max	1,800	1,160
Nonzero demand, mean±std	159.5 ± 242.3	11.6 ± 29.3
Total demand per snapshot	890,242	215,426
Active pairs per snapshot	5,583	18,595
Active-pair union (trace)	6,334	125,949

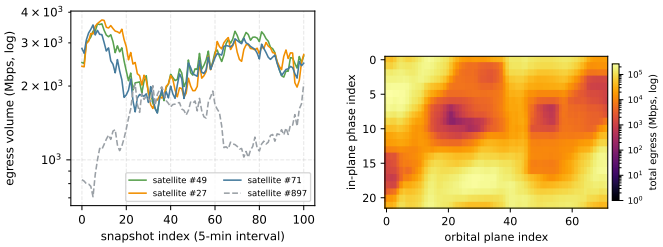

Fig. 4. Traffic characteristics of the trace (real data). Left: egress volume of representative satellites over time. Right: total egress per satellite on the orbital grid — traffic concentrates over populated regions.

5.2 Overall Performance

Figure 6 reports the satisfied demand ratio as observability ρ decreases, with detailed numbers in table 6. SiTE degrades gracefully, retaining [TODO: XX]% of the fully-observed optimum even at $\rho = 0.1$, whereas zero-filling and mean-interpolation drop sharply as more demands become unmeasured. The gap widens as ρ shrinks, confirming that mask embeddings and temporal recovery matter most when information is scarcest. Figure 5 further shows the per-snapshot distribution at $\rho = 0.3$: SiTE is not only better on average but also markedly more stable across snapshots.

5.3 Ablation Study

Figure 8 isolates each component at $\rho = 0.3$. Removing the learnable mask embedding causes the largest drop, followed by the gated message passing and the temporal encoder; disabling all three recovers the zero-fill baseline. Figure 7 extends the ablation across observability levels: the contribution of every module grows as ρ decreases, and the mask embedding dominates in the extreme-sparsity regime.

The temporal module also stabilizes training: fig. 9 shows that the history-augmented model converges to a higher and steadier validation ratio than its single-snapshot counterpart.

5.4 Zero-Retraining under Demand and Topology Drift

Demand-set drift. We train on the demand-pair union of the training slice (6,320 pairs) and, at test time, rebuild the graph from the test slice's own union (6,044 pairs, including pairs never seen in training) while keeping the weights frozen. Table 7 shows that this incurs negligible loss relative to the oracle setting where the demand set is built from all snapshots, and fig. 10 confirms the gap stays small on every test snapshot, validating train-once, zero-retraining deployment under demand-set drift.

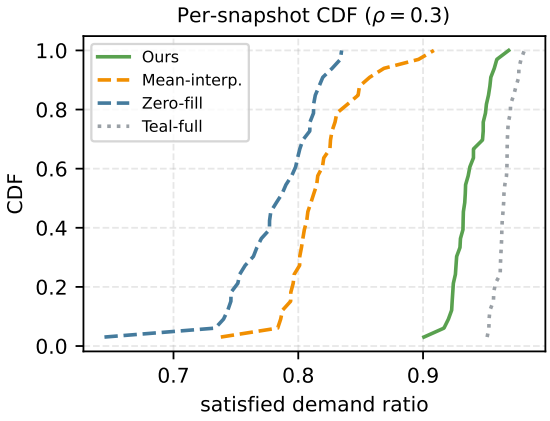

Fig. 5. CDF of per-snapshot satisfied ratio at $\rho = 0.3$ (placeholder data).

TABLE 6
Satisfied demand ratio (mean±std over 3 seeds, placeholder data). "—" marks settings that do not apply.

ρ	Zero-fill	Mean-interp.	**Ours**	Teal-full
1.0	—	—	—	[TODO: .XX]
0.7	[TODO: .XX]	[TODO: .XX]	[TODO: .XX]	—
0.5	[TODO: .XX]	[TODO: .XX]	[TODO: .XX]	—
0.3	[TODO: .XX]	[TODO: .XX]	[TODO: .XX]	—
0.1	[TODO: .XX]	[TODO: .XX]	[TODO: .XX]	—

Topology drift. Constellation topologies also change — ISLs fail or are re-planned as satellites move. We emulate this by testing the model, trained on the nominal +Grid topology, on perturbed variants with $5\%/10\%$ of ISLs removed (candidate paths and the bipartite graph are rebuilt at test time; weights stay frozen), and additionally with k random link failures injected at test time. As table 8 shows, SiTE degrades marginally under both perturbations, substantiating Proposition 2: the graph is a runtime input, not a parameter.

5.5 Runtime and Scalability

Demand pruning shrinks the modeled problem from 2.5M pairs to ∼6.3K (∼400×), bringing per-snapshot inference to [TODO: XX] ms on a single GPU — within the control budget of a rapidly changing constellation, and comparable to the millisecond-scale latency reported by SaTE [12]. Figure 11 contrasts SiTE with an LP solver and Teal-full: learned inference is three orders of magnitude faster than solving, and the temporal module adds only marginal latency. Modeling the full demand set without pruning is infeasible under the memory budget, as the index tensors alone would exceed [TODO: XX] GB.

Control-loop budget. Inference is only part of the control loop: collecting sparse measurements and disseminating split ratios traverse ground–satellite and inter-satellite links, adding propagation delays on the order of a few tens of milliseconds in LEO. Millisecond-scale inference keeps the end-to-end loop dominated by propagation rather than computation; moreover, since the trained model has fewer than 5,000 parameters (section 4.7), it can also be replicated onboard to shorten the dissemination path.

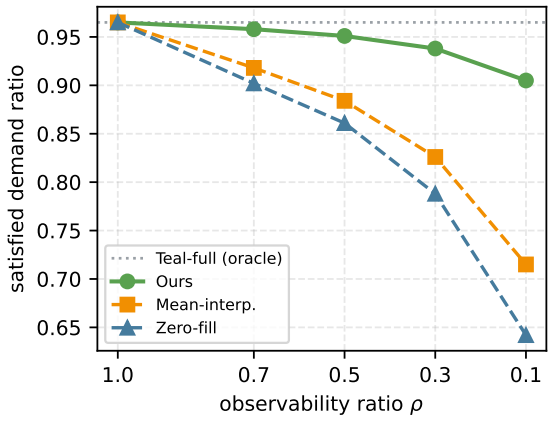

Fig. 6. Satisfied demand ratio versus observability ratio ρ (placeholder data). SiTE stays close to the fully-observed oracle while baselines degrade sharply.

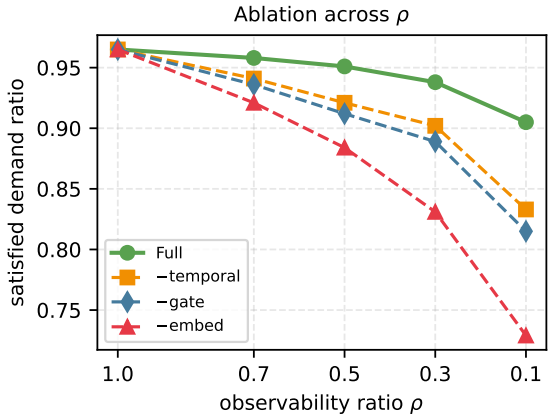

Fig. 7. Ablation across observability ratios (placeholder data). Module contributions grow as observations become scarcer.

5.6 Sensitivity Analysis

Measurement granularity. Figure 12 compares flow- and node-level sampling at equal ρ. Node-level missing is structural — entire source rows disappear (fig. 2) — and hurts the zero-fill baseline substantially more, while SiTE remains robust thanks to spatial message passing from links shaped by observed traffic.

History length. Figure 13 sweeps L: quality saturates around L=[$TODO : X$] while latency grows linearly, so we adopt L=3 by default.

Load sensitivity. Figure 14 varies ISL capacity: the advantage of SiTE widens under high load, where misallocating unmeasured demands is most costly.

Traffic model. On the GDP-based trace — thinner flows spread over ∼20× more demand pairs (table 5) — the pruned demand set grows to ∼126K pairs, stressing the scalability of the pipeline; SiTE retains [TODO: XX]% of the fully-observed performance at $\rho = 0.3$, showing that the sparse-aware design does not depend on a particular traffic concentration pattern.

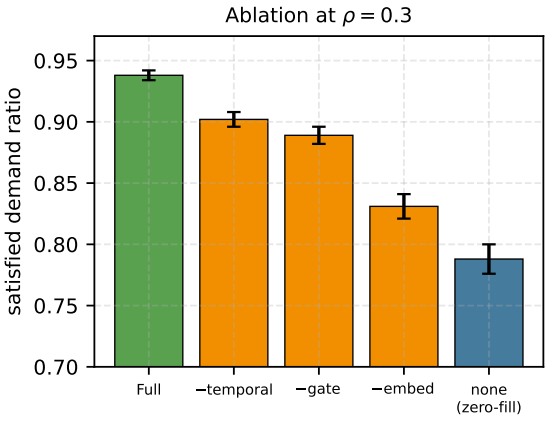

Fig. 8. Component ablation at $\rho = 0.3$ (placeholder data). Each sparse-aware module contributes to the final performance.

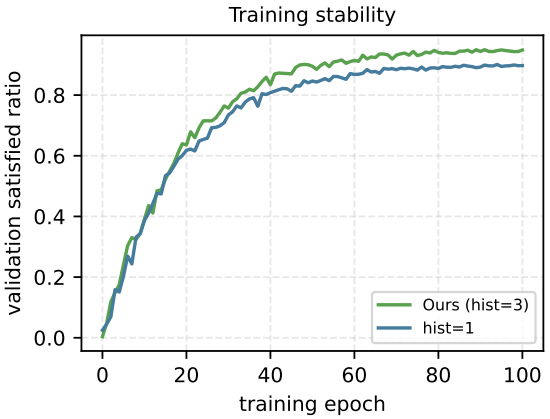

Fig. 9. Validation satisfied ratio during training (placeholder data). Temporal fusion improves both final quality and stability.

6 Conclusion

This paper presented SiTE, the first learning-based traffic engineering system for LEO satellite constellations that operates directly on sparse traffic observations. SiTE resolves the ambiguity of unmeasured demands with learnable mask embeddings, protects network-state estimates with mask-aware gated message passing, and recovers missing information from observation history through temporal-spatial cross-attention — all trained end-to-end against the TE objective without an explicit completion stage. Demand pruning and size-invariant weights let a model trained once serve drifting topologies and demand sets with zero retraining. On a 1,584-satellite Starlink-like constellation with real traffic dynamics, SiTE keeps the realized maximum link utilization within [TODO: XX]% of the fully-observed optimum with only [TODO: XX]% observability — and retains [TODO: XX]% of the optimum under the throughput objective — outperforming zero-filling and two-stage baselines by [TODO: XX–XX]%, at [TODO: XX] ms inference per snapshot.

Three directions remain open. First, our observation masks are static within a trace; real measurement availability fluctuates with ground-contact windows and onboard

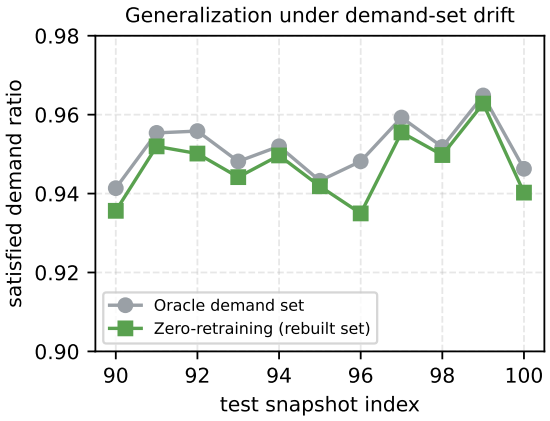

Fig. 10. Per-snapshot satisfied ratio on the test slice (placeholder data): weights trained on the training-slice demand set are reused on the rebuilt test-slice set with negligible loss.

TABLE 7
Zero-retraining generalization (placeholder data).

Setting	Train set	Test set	Satisfied ratio
Oracle (full set)	6,334	6,334	[TODO: 0.XX]
Zero-retraining (split)	6,320	6,044	[TODO: 0.XX]

TABLE 8
Topology-drift zero-retraining (placeholder data).

Test topology	ISLs	Satisfied ratio
Nominal (training topology)	6,336	[TODO: 0.XX]
5% ISLs removed	6,020	[TODO: 0.XX]
10% ISLs removed	5,704	[TODO: 0.XX]
k=50 random link failures	6,336	[TODO: 0.XX]

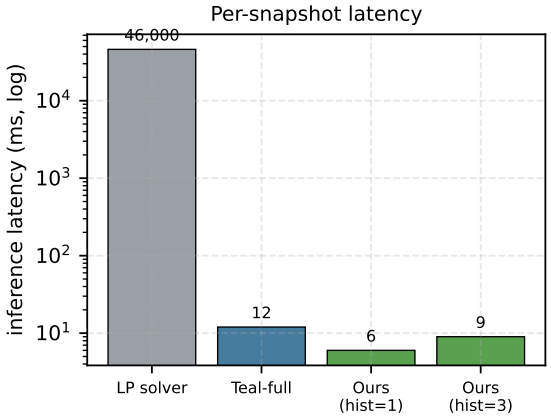

Fig. 11. Per-snapshot inference latency (log scale, placeholder data).

load, motivating time-varying masks and measurement scheduling — deciding *which* demands to measure next — as a joint control problem. Second, our input-side modules are orthogonal to the training paradigm: combining them with imitation learning [26] where post-hoc complete traces are available, or with hybrid schemes, may improve sample efficiency. Third, the path templates of +Grid constellations suggest an $O(1)$ displacement-indexed path lookup that we currently approximate with per-set path caching; a full implementation would further cut graph-rebuild latency. We also plan to validate on measured (rather than synthesized) satellite traffic as such datasets become available.

References

[1] M. Handley, "Delay is not an option: Low latency routing in space," in *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)*, 2018.

[2] D. Bhattacherjee and A. Singla, "Network topology design at 27,000 km/hour," in *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2019.

[3] G. Giuliari, T. Klenze, M. Legner, D. Basin, A. Perrig, and A. Singla, "Internet backbones in space," *ACM SIGCOMM Computer Communication Review*, vol. 50, no. 1, 2020.

[4] S. Kassing, D. Bhattacherjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the "internet from space" with hypatia," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.

[5] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at starlink performance," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2022.

[6] S. Jain *et al.*, "B4: Experience with a globally-deployed software defined WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013, tODO: full author list.

[7] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, "Achieving high utilization with software-driven WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013.

[8] C.-Y. Hong *et al.*, "B4 and after: Managing hierarchy, partitioning, and asymmetry for availability and scale in google's software-defined WAN," in *Proceedings of the ACM SIGCOMM 2018 Conference*, 2018, tODO: full author list.

[9] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, and M. Yu, "Teal: Learning-accelerated optimization of WAN traffic engineering," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 378–393.

[10] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, and A. Tamar, "DOTE: Rethinking (predictive) WAN traffic engineering," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1557–1581.

[11] A. A. AlQiam, Y. Yao, Z. Wang, S. S. Ahuja, Y. Zhang, S. G. Rao, B. Ribeiro, and M. Tawarmalani, "Transferable neural WAN TE for changing topologies," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 86–102.

[12] H. Wu, Y. Han, M. Rajpal, Q. Zhang, and J. Wang, "SaTE: Low-latency traffic engineering for satellite networks," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 896–916.

[13] Y. Guo, Z. Huang, M. Ding, and H. Luo, "An end-to-end learning approach for traffic engineering with sparse traffic measurements," *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 5448–5463, 2026, tODO: verify author list against the published version.

[14] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proceedings of IEEE INFOCOM 2000*, 2000.

[15] H. Wang, H. Xie, L. Qiu, Y. R. Yang, Y. Zhang, and A. Greenberg, "COPE: Traffic engineering in dynamic networks," in *Proceedings of the ACM SIGCOMM 2006 Conference*, 2006.

[16] P. Kumar, Y. Yuan, C. Yu, N. Foster, R. Kleinberg, P. Lapukhov, C. L. Lim, and R. Soulé, "Semi-oblivious traffic engineering: The road not taken," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 157–170.

[17] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, P. Bailis, and M. Zaharia, "Contracting wide-area network topologies to solve flow problems quickly," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, tODO: verify author list.

[18] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, and M. Zaharia, "Solving large-scale granular resource allocation problems efficiently with POP," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, 2021, tODO: verify author list.

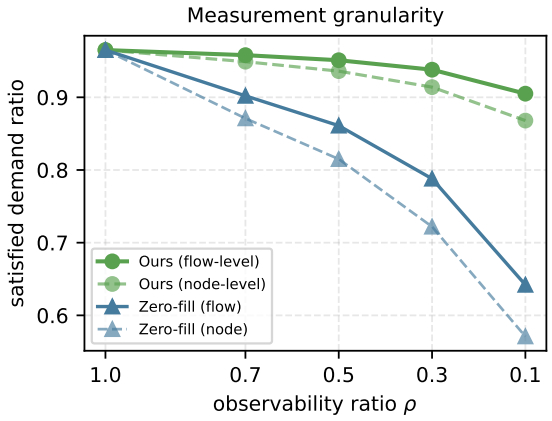

Fig. 12. Flow- versus node-level sparse sampling (placeholder data).

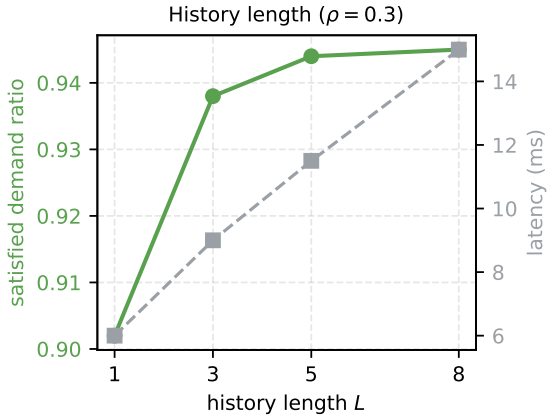

Fig. 13. History-length sweep at $\rho = 0.3$ (placeholder data): quality saturates while latency grows linearly.

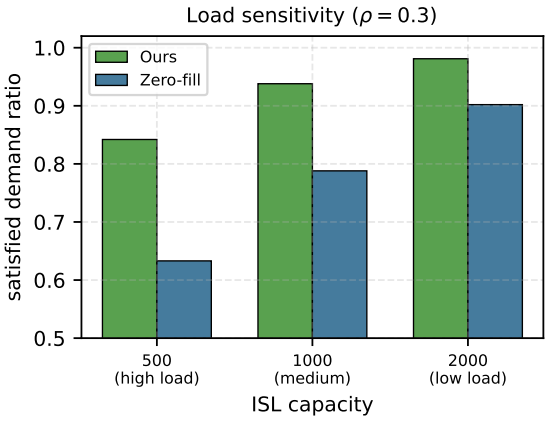

Fig. 14. Sensitivity to ISL capacity at $\rho = 0.3$ (placeholder data).

[19] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, 2017.

[20] X. Liu *et al.*, "FIGRET: Fine-grained robustness-enhanced traffic engineering," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, tODO: verify author list and pages.

[21] D. Wu *et al.*, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, tODO: verify author list and pages.

[22] Y. Lu, Y. Zhao, F. Sun, and H. Li, "Virtual topology for LEO satellite networks based on earth-fixed footprint mode," *IEEE Communications Letters*, vol. 17, no. 2, pp. 357–360, 2013, tODO: verify author list.

[23] M. Hu, M. Xiao, W. Xu, T. Deng, Y. Dong, and K. Peng, "Traffic engineering for software-defined LEO constellations," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 5090–5103, 2022, tODO: verify author name spellings.

[24] L. Wei, T.-T. Le, Y. Ji, M. Wang, Y. Liu, Y. Wang, and J. C. Lui, "Towards efficient traffic engineering via distributed optimization in large-scale LEO constellation," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2024, pp. 353–358, tODO: verify co-author name spellings.

[25] X. Liu, Z. Zhang, X. Qin, H. Zhou, and L. Zhao, "FlexSATE: Flexible and distributed traffic engineering with supervised learning in ultra-dense low-earth-orbit satellite networks," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 4466–4471, tODO: verify author name spellings.

[26] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, C. Ying, T. Liu, S. Yue, and H. Jin, "Scalable traffic allocation in dynamic networks via end-to-end imitation learning," *IEEE/ACM Transactions on Networking*, vol. 34, 2026.

[27] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, T. Liu, and H. Jin, "Learning to accelerate traffic allocation over large-scale networks," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025.

[28] Y. Vardi, "Network tomography: Estimating source-destination traffic intensities from link data," *Journal of the American Statistical Association*, vol. 91, no. 433, 1996.

[29] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale IP traffic matrices from link loads," in *Proceedings of ACM SIGMETRICS 2003*, 2003.

[30] Y. Zhang, M. Roughan, W. Willinger, and L. Qiu, "Spatio-temporal compressive sensing and internet traffic matrices," in *Proceedings of the ACM SIGCOMM 2009 Conference*, 2009.

[31] C. Huo, D. Jin, Y. Li, D. He, Y.-B. Yang, and L. Wu, "T2-GNN: Graph neural networks for graphs with incomplete features and structure via teacher-student distillation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[32] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[34] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[35] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, 2011.

[36] G. Giuliari, T. Ciussani, A. Perrig, and A. Singla, "ICARUS: Attacking low earth orbit satellite networks," in *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021, pp. 317–331.